



LAPORAN PRAKTIKUM

Mata Kuliah Pengembangan Aplikasi Web II

TUGAS 1: RESTful API Murni dengan Express.js

Topik 18: Kampus: Alumni

Disusun Oleh:

Nama : Kirana Cinta Mentari

NIM : 2428240078

Kelas : SI5B

Nomor Topik : 18

Nama Topik : Kampus: Alumni

Dosen Pengampu: Nur Rachmat, M.Kom.

PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027

1. Tujuan Praktikum

Praktikum ini bertujuan agar mahasiswa mampu:

1. Membuat project berbasis Express.js yang dapat dijalankan secara local maupun di vercel, dengan struktur satu file app.js yang terseusun urut dan berkomentar, serta repository Github dengan commit bertahap.
2. Membuat routing RESTFUL untuk resource alumni dengan enam endpoint (CRUD dan filter query string), penamaan URL sesuai dengan konvensi, dan handler 404.
3. Menggunakan method HTTP GET, POST, PUT, dan DELETE sesuai fungsinya, lengkap dengan validasi field wajib, id otomatis, serta penanganan kasus 404 dan 400.
4. Menghasilkan response murni JSON dengan format dan status kode yang tepat seperti 200 (berhasil), 201 (berhasil membuat data baru), 400 (salah atau tidak valid), 404 (tidak ditemukan) tanpa menggunakan res.send() teks/HTML.

2. Dasar Teori

2.1. REST dan Resource

REST (Representational State Transfer) Adalah gaya arsitektur untuk sistem terdistribusi (fielding, 2000). Dalam REST, data dipandang sebagai resource yang diidentifikasi oleh URL, dan klien mengelolanya melalui method HTTP. Penamaan resource umumnya berupa kata beda jamak, huruf kecil, tanpa kata kerja di URL, sehingga /alumni lebih tepat daripada /getAlumni. Layanan yang menerapkan prinsip ini disebut RESTful API.

2.2. Method HTTP

Method HTTP menyatakan operasi yang diminta klien terhadap resource (Fielding dkk., 2022). Dengan menggunakan empat method, yaitu GET dipakai untuk membaca data tanpa mengubahnya, POST untuk membuat data baru pada resource, PUT untuk mengganti seluruh data pada resource tertentu, DELETE untuk menghapus data pada resource tertentu. Dari keempat method tersebut membentuk operasi CRUD (create, read, update, delete).

2.3. HTTP Status Code

Status code menunjukkan hasil pemrosesan request (MDN Web Docs, 2025). Kode 200 (OK) menandakan permintaan berhasil, 201 (Created) menandakan data baru berhasil dibuat, 400 (Bad Request) menandakan permintaan tidak valid, dan 404 (Not Found) menandakan resource tidak ditemukan.

2.4. JSON

JSON (JavaScript Object Notation) adalah format pertukaran data berbasis teks yang ringan dan mudah dibaca oleh manusia maupun mesin (Bray, 2017). Data disusun dalam

pasangan nama dan nilai. Pada RESTful API, JSON menjadi format response sehingga tidak ada halaman HTML yang dikirim ke klien.

2.5. Express.js

Express.js adalah framework web minimalis untuk Node.js yang menyediakan routing dan middleware (Express.js, 2025). Routing dibuat dengan `app.get()`, `app.post()`, `app.put()`, dan `app.delete()`. Middleware `express.json()` digunakan untuk membaca body request berformat JSON ke dalam `req.body`, `req.params` untuk parameter pada URL, dan `req.query` untuk query string.

3. Alat dan Bahan

No	Nama Alat / Bahan	Versi / Keterangan
1	Node.js (LTS)	Versi: v24.15.0
2	npm	Versi: 12.0.2
3	Express.js	Versi 5.2.1 (framework web)
4	nodemon	Versi 3.1.14 (devDependency, restart otomatis)
5	Editor kode	Visual Studio Code
6	Postman	Alat pengujian endpoint API
7	Git	Versi 2.56.0.windows.1
8	GitHub	Akun GitHub untuk repository publik
9	Vercel	Akun Vercel untuk deploy

4. Langkah Praktikum

4.1. Inisialisasi Project

Project dibuat di folder `tugas1-restful-2428240078`. Perintah `npm init -y` membuat `package.json`, lalu `express` dipasang sebagai dependency dan `nodemon` sebagai

devDependency. Pada package.json, entry point diatur ke app.js dan ditambahkan script start serta dev.

```
() package.json > {} dependencies > abc express
1  {
2    "name": "tugas1-restful-2428240078",
3    "version": "1.0.0",
4    "description": "Tugas 1 RESTful API Express.js - Topik 18 - Kampus: Alumni",
5    "main": "app.js",
6    >Debug
7    "scripts": {
8      "start": "node app.js",
9      "dev": "nodemon app.js"
10   },
11   "keywords": [],
12   "type": "commonjs",
13   "dependencies": {
14     "express": "^5.2.1"
15   },
16   "devDependencies": {
17     "nodemon": "^3.1.14"
18   }
19 }
```

```
PS C:\Users\cinta\tugas1-restful-2428240078> npm install

up to date, audited 95 packages in 859ms

33 packages are looking for funding
  run `npm fund` for details

3 high severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
```

Gambar 1. Package.json (main dan script) dan terminal saat npm install dijalankan

4.2. Middleware dan Data Awal

File app.js dimulai dengan impor Express, pemasangan middleware express.json(), array data awal, dan variable nextId untuk id berikutnya.

```
1  const express = require('express');
2
3  const app = express();
4  const PORT = process.env.PORT || 3000;
5
6  // Wajib didaftarkan sebelum route yang membaca req.body
7  app.use(express.json());
8  // data awal (minimal 3)
9  let alumni = [
10   { id: 1, nim: "2020110045", nama: "Farhan Akbar", prodi: "Sistem Informasi", tahunLulus: 2024, pekerjaan: "Business Analyst" },
11   { id: 2, nim: "2019110012", nama: "Siti Rahma", prodi: "Teknik Informatika", tahunLulus: 2023, pekerjaan: "Web Developer" },
12   { id: 3, nim: "2020110078", nama: "Dimas Prakoso", prodi: "Sistem Informasi", tahunLulus: 2024, pekerjaan: "Data Analyst" },
13 ];
14 let nextId = 4;
15
```

Gambar 2. Bagian awal app.js (express.json dan data awal)

4.3. Route GET /

Route awal membalas JSON yang berisi nama, NIM, nomor Topik, endpoint, tanpa `res.send()` teks.

```
// GET /
app.get("/", (req, res) => {
  res.json({
    nama: "Kirana Cinta Mentari",
    nim: "2428240078",
    nomorTopik: 18,
    endpoints: "/alumni",
  });
});
```

Gambar 3. Kode route GET /

4.4. Route GET (Semua Data, Satu Data, dan Filter)

GET /alumni mengembalikan seluruh data. Jika terdapat query string tahunLulus, data difilter dengan `filter()` dan hasilnya berupa array (boleh kosong). GET /alumni/:id mencari satu data dengan `find()`, dan membalas 404 jika tidak ditemukan. Response GET berupa data langsung tanpa status dan message.

```
26 // GET /alumni menampilkan seluruh data alumni
27 // GET /alumni
28 app.get('/alumni', (req, res) => {
29   const { tahunLulus } = req.query;
30
31   // jika ada filter, kembalikan data yang cocok saja
32   if (tahunLulus) {
33     const hasil = alumni.filter(a => a.tahunLulus === parseInt(tahunLulus));
34     return res.status(200).json(hasil);
35   }
36
37   // jika tidak ada filter, kembalikan semua data
38   res.status(200).json(alumni);
39 });
40
41 // GET /alumni/:id menampilkan data alumni berdasarkan id
42 app.get('/alumni/:id', (req, res, next) => {
43   const id = parseInt(req.params.id);
44   const data = alumni.find(a => a.id === id);
45
46   if (!data) {
47     return res.status(404).json({
48       status: 'error',
49       message: `Data dengan id ${id} tidak ditemukan`,
50       data: null,
51     });
52   }
53   res.status(200).json(data);
54 });
```

Gambar 4. Kode route GET di app.js

4.5. Route POST dan Validasi

Post Membaca body JSON, memvalidasi field wajib (NIM, nama, program studi, tahunLulus), dan memastikan tahunLulus bertipe angka. Jika tidak valid, server membalas 400. Jika valid, id dibuat otomatis dengan nextId++ dan server membalas 201.

```
// POST /alumni
// Body: { "nim": "2020110045", "nama": "Farhan Akbar", "prodi": "Sistem Informasi", "tahunLulus": 2024, "pekerjaan": "Business Analyst" }
app.post('/alumni', (req, res) => {
  const { nim, nama, prodi, tahunLulus, pekerjaan } = req.body;

  // validasi: field wajib kosong -> 400
  if (!nim || !nama || !prodi || tahunLulus === undefined || tahunLulus === null || tahunLulus === '') {
    return res.status(400).json({
      status: 'error',
      message: 'fields wajib diisi',
      data: null,
    });
  }

  // tahunLulus harus berupa angka
  if (typeof tahunLulus !== 'number') {
    return res.status(400).json({
      status: 'error',
      message: 'tahunLulus harus berupa angka!',
      data: null,
    });
  }

  const baru = { id: nextId++, nim, nama, prodi, tahunLulus, pekerjaan: pekerjaan || '' };
  alumni.push(baru);

  // berhasil -> 201 + data yang baru dibuat
  res.status(201).json({
    status: 'success',
    message: 'Data berhasil ditambahkan',
    data: baru,
  });
});
```

Gambar 5. Kode route POST dan Validasi pada app.js

4.6. Route PUT

PUT mengganti seluruh field data dengan id tertentu. Server membalas 404 jika id tidak ada dan 400 jika field wajib kosong atau tahunLulus bukan angka (validasi sama seperti POST). Jika berhasil, server membalas 200 atau berhasil.

```
app.put('/alumni/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const index = alumni.findIndex(a => a.id === id); // mencari index array

  // jika id tidak ada -> 404
  if (index === -1) {
    return res.status(404).json({
      status: 'error',
      message: `Data dengan id ${id} tidak ditemukan`,
      data: null,
    });
  }

  const { nim, nama, prodi, tahunLulus, pekerjaan } = req.body;

  // validasi: field wajib kosong -> 400
  if (!nim || !nama || !prodi || tahunLulus === undefined || tahunLulus === null || tahunLulus === '') {
    return res.status(400).json({
      status: 'error',
      message: 'fields wajib diisi',
      data: null,
    });
  }

  // tahunLulus harus berupa angka
  if (typeof tahunLulus !== 'number') {
    return res.status(400).json({
      status: 'error',
      message: 'tahunLulus harus berupa angka!',
      data: null,
    });
  }

  // ganti penuh semua field (id tetap)
  alumni[index] = { id, nim, nama, prodi, tahunLulus, pekerjaan: pekerjaan || '' };

  res.status(200).json({
    status: 'success',
    message: 'Data berhasil diubah',
    data: alumni[index],
  });
});
```

Gambar 6. Kode route PUT pada app.js

4.7. Route DELETE

DELETE menghapus data dengan splice(). Jika id tidak ditemukan, server membalas 404 (tidak ditemukan). Jika berhasil, server membalas 200 dengan pesan yang menyebut id yang dihapus dan data bernilai null.

```
// DELETE /alumni/3
app.delete('/alumni/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const index = alumni.findIndex((a) => a.id === id);

  // jika id tidak ada -> 404
  if (index === -1) {
    return res.status(404).json({
      status: 'error',
      message: `Data tidak ditemukan`,
      data: null,
    });
  }

  alumni.splice(index, 1); // hapus dari array

  // berhasil -> 200 + JSON berisi pesan, data null
  res.status(200).json({
    status: 'success',
    message: `Data alumni dengan id ${id} berhasil dihapus`,
    data: null,
  });
});
```

Gambar 7. Kode route DELETE pada app.js

4.8. Middleware 404 dan Penanganan Error

Middleware catch-all 404 diletakkan setelah semua route agar URL yang tidak terdaftar dibalas JSON, bukan HTML bawaan Express. Error handler dengan empat parameter menangani body JSON yang rusak agar tetap dibalas JSON.

```
// error handler: body JSON yang rusak tetap dibalas JSON (bukan HTML bawaan Express)
app.use((err, req, res, next) => {
  res.status(400).json({
    status: 'error',
    message: 'Body json rusak atau tidak valid',
    data: null,
  });
});

// middleware catch-all 404 (paling bawah, setelah semua route)
app.use((req, res) => {
  res.status(404).json({
    status: 'error',
    message: 'Endpoint tidak ditemukan',
    data: null,
  });
});
```

Gambar 8. Kode middleware 404 dan error handler

4.9. Persiapan Deploy ke Vercel

Pada tahap persiapan deployment ke Vercel, `app.listen()` digunakan hanya saat aplikasi dijalankan secara lokal. Untuk deployment, aplikasi diekspor menggunakan `module.exports = app` agar dapat dijalankan oleh Vercel sebagai serverless function. Selain itu, file `vercel.json` ditambahkan pada root project untuk mengatur konfigurasi deployment dan routing aplikasi. File `gitignore` berisi `node_modules/` dan `env` agar kedua item tersebut tidak ikut ter-upload.

```
app.listen(PORT, () =>
|   console.log(`Server berjalan di http://localhost:${PORT}`)
|);
module.exports = app;
```

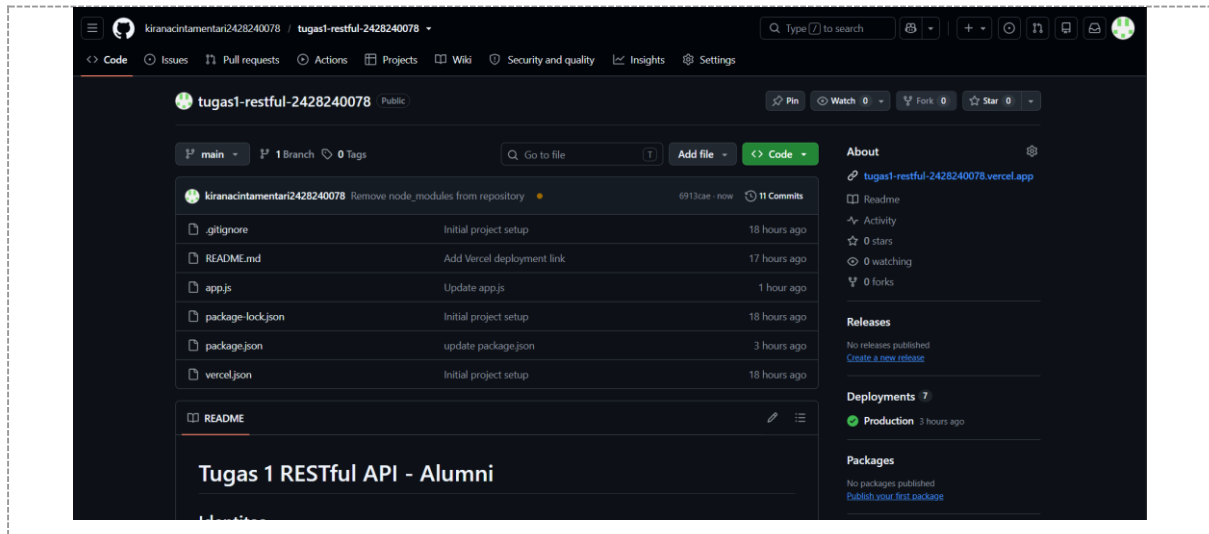
```
{
  "version": 2,
  "builds": [
    {
      "src": "app.js",
      "use": "@vercel/node"
    }
  ],
  "routes": [
    {
      "src": "/(.*)",
      "dest": "app.js"
    }
  ]
}
```

Gambar 9. Bagian akhir app.js dan isi vercel.json

4.10. Push ke GitHub

Pada tahap push ke GitHub, project diunggah ke repository sebagai tempat penyimpanan dan pengelolaan kode. Gambar menunjukkan repository GitHub yang berisi file-file project seperti app.js, package.json, package-lock.json, README.md, dan vercel.json. Ini menandakan bahwa seluruh kode aplikasi telah berhasil di-*push*

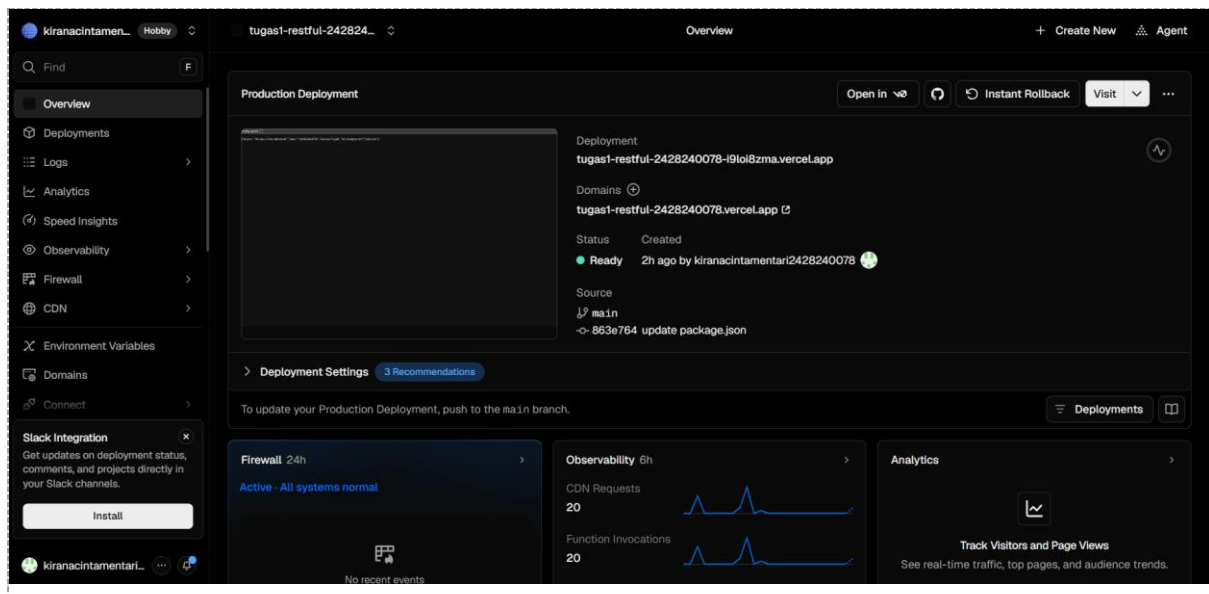
dan tersimpan pada repository sehingga dapat digunakan sebagai sumber kode untuk proses deployment ke Vercel.



Gambar 10. Halaman repository GitHub tugas1-restful-2428240078

4.11. Deploy ke Vercel

Repository diimpor melalui dashboard vercel (add New, Project), lalu deploy. Setiap push ke branch main memicu deploy ulang otomatis. Deploy berstatus ready dan dapat diakses melalui <https://tugas1-restful-2428240078.vercel.app/>.



Gambar 11. Dashboard Vercel dengan status Ready

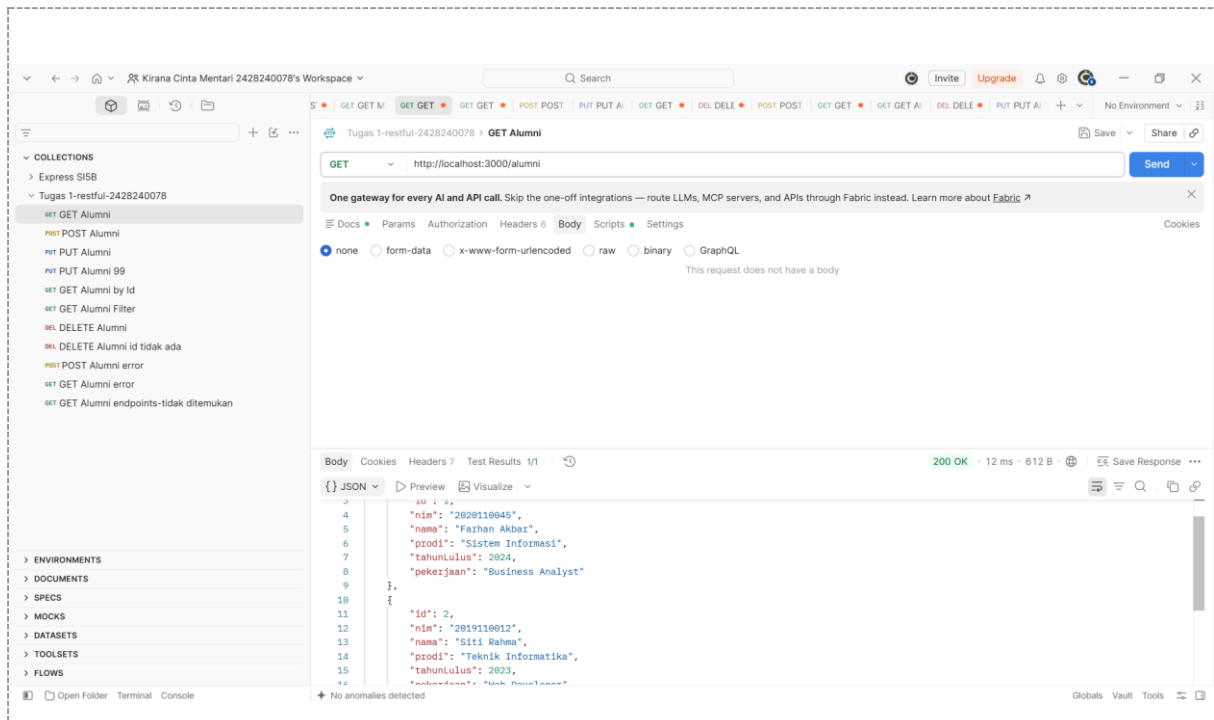
5. Hasil Pengujian

Pengujian dilakukan dengan Postman pada URL Vercel: [<https://tugas1-restful-2428240078.vercel.app/>]. Tabel berikut memuat Request pengujian beserta satu skenario tambahan untuk middleware catch-all.

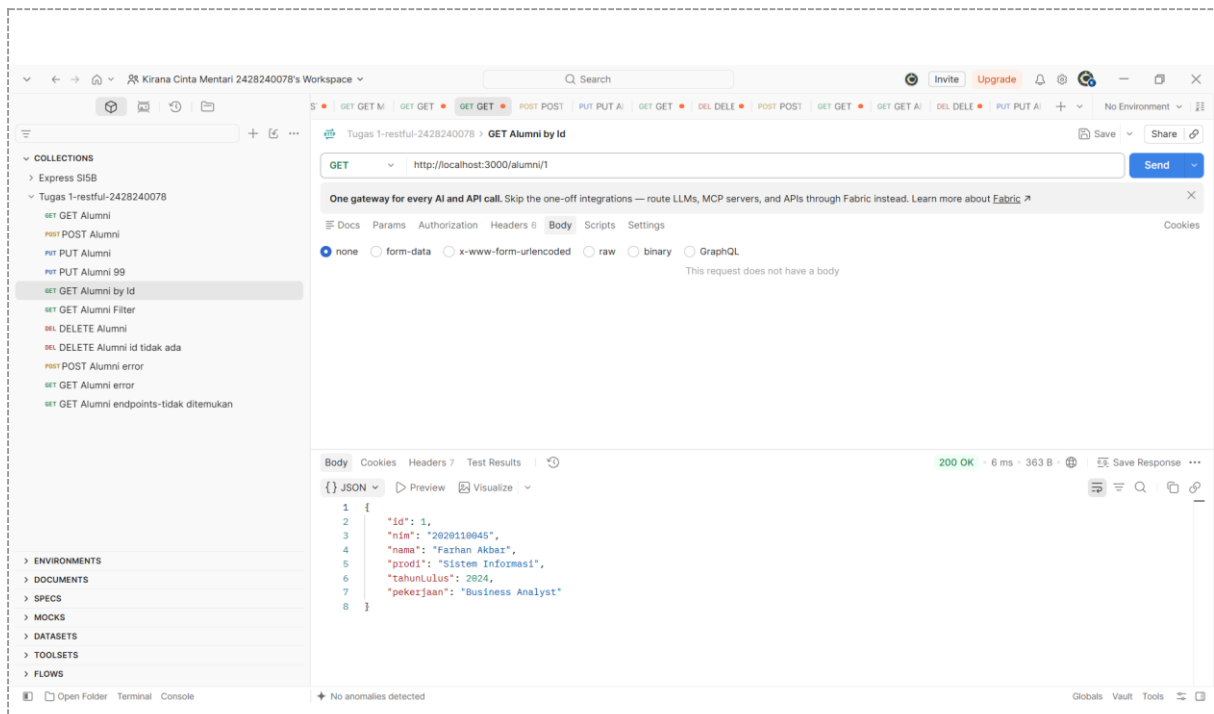
No	Method	Endpoint	Data request	Status diharapkan	Status hasil	Keterangan
1	GET	/alumni	—	200	200	Berhasil menampilkan seluruh data alumni.
2	GET	/alumni/1	—	200	200	Berhasil menampilkan data alumni dengan Id 1.
3	GET	/alumni/99	—	404	404	Berhasil memberikan respons 404 karena data dengan Id 99 tidak ditemukan.
4	GET	/alumni?tahunLulus=2024	—	200	200	Berhasil menampilkan data alumni berdasarkan tahun lulus 2024.
5	POST	/alumni	Body lengkap	201	201	Berhasil menambahkan data alumni baru dengan data yang lengkap.

No	Method	Endpoint	Data request	Status diharapkan	Status hasil	Keterangan
6	POST	/alumni	Field wajib kosong	400	400	Berhasil menolak data karena terdapat field wajib yang kosong.
7	PUT	/alumni/1	Body lengkap	200	200	Berhasil memperbarui data alumni.
8	PUT	/alumni/99	Body lengkap	404	404	Berhasil memberikan respons 404 karena data alumni dengan Id 99 tidak ditemukan.
9	DELETE	/alumni/3	—	200	200	Berhasil menghapus data alumni dengan ID 3.
10	DELETE	/alumni/99	—	404	404	Berhasil memberikan respons 404 karena data alumni dengan ID 99 tidak ditemukan.
11	GET	/abc (catch-all)	—	404	404	Berhasil menangani endpoint yang tidak tersedia dengan respons 404.

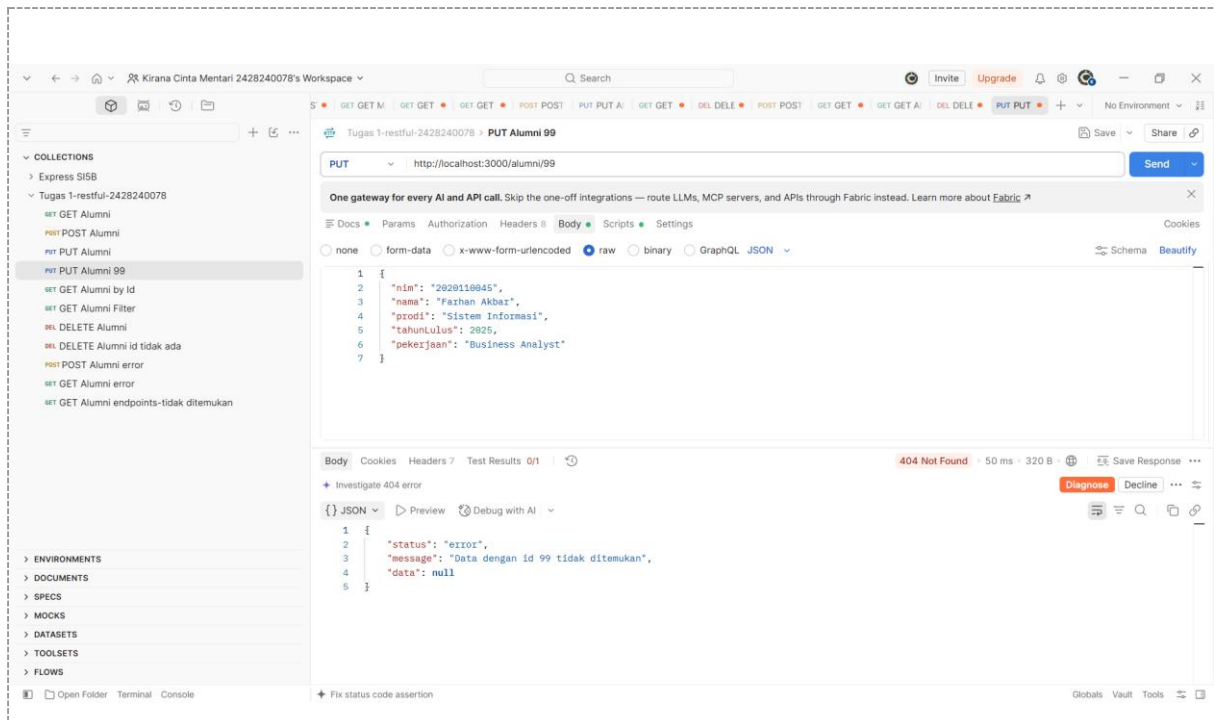
Screenshot request, response, dan status code setiap pengujian ditampilkan sebagai berikut:



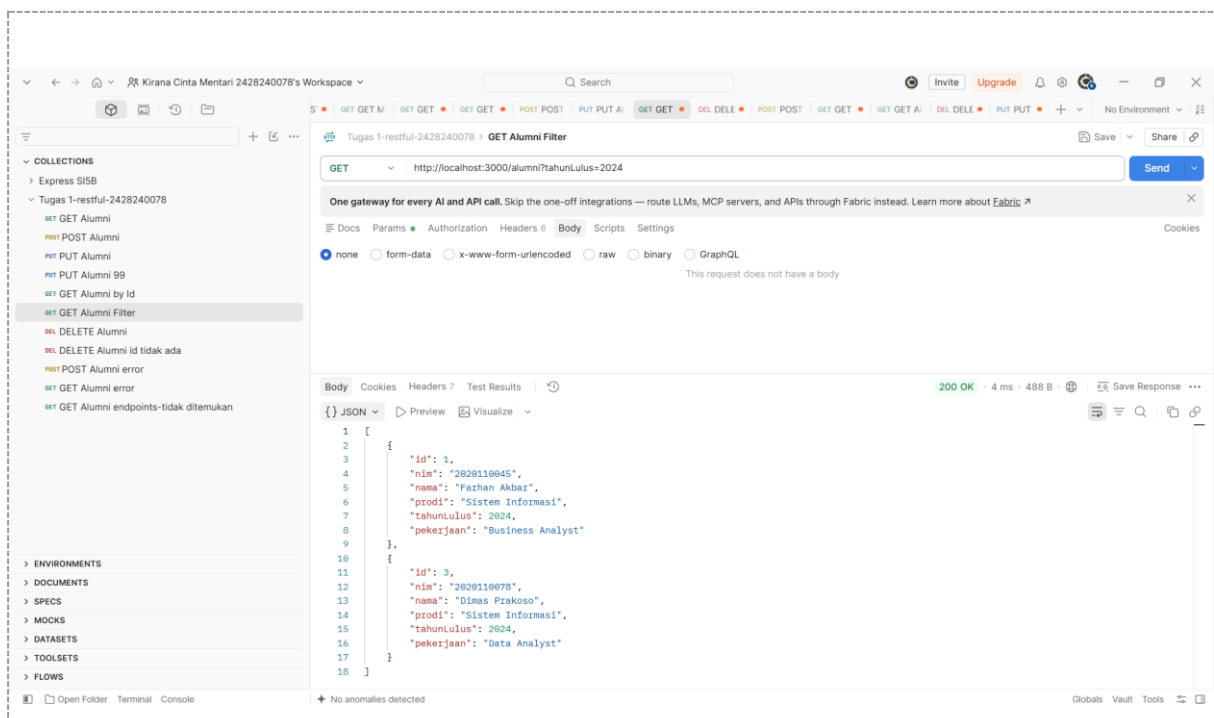
Gambar 5.1. Request 1: GET /alumni (200)



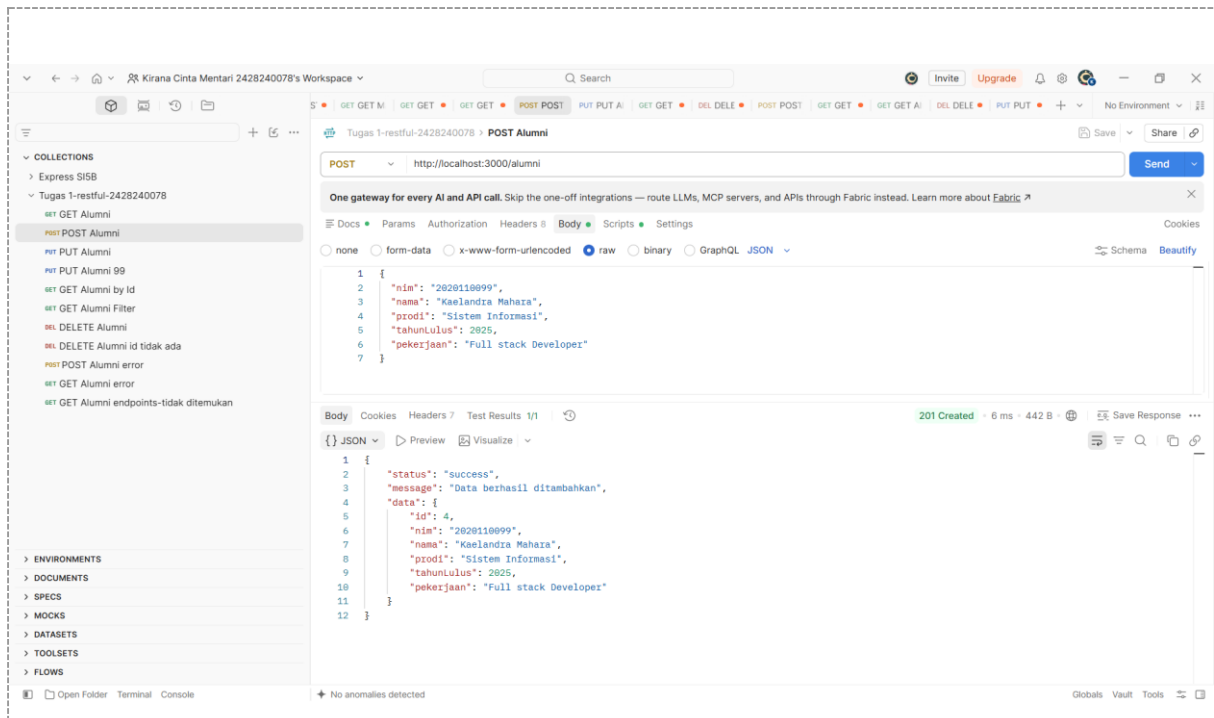
Gambar 5.2. Request 2: GET /alumni/1 (200)



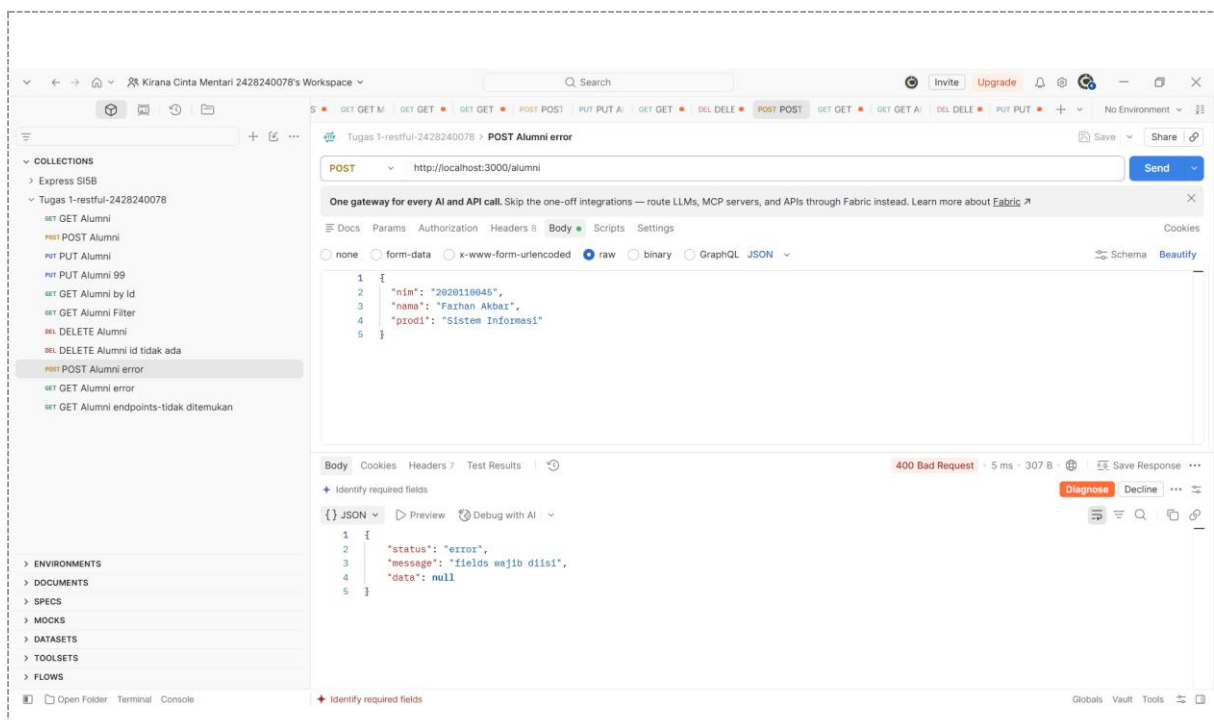
Gambar 5.3. Request 3: GET /alumni/99 (404)



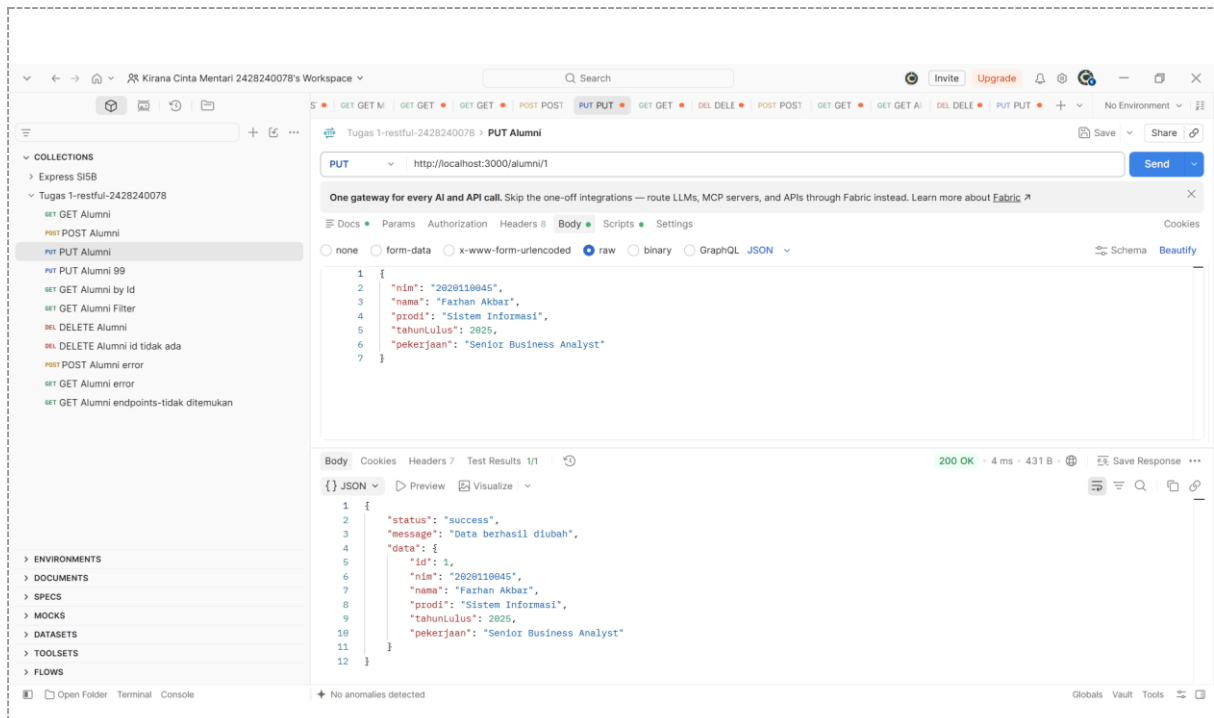
Gambar 5.4. Request 4: GET /alumni?tahunLulus=2024 (200)



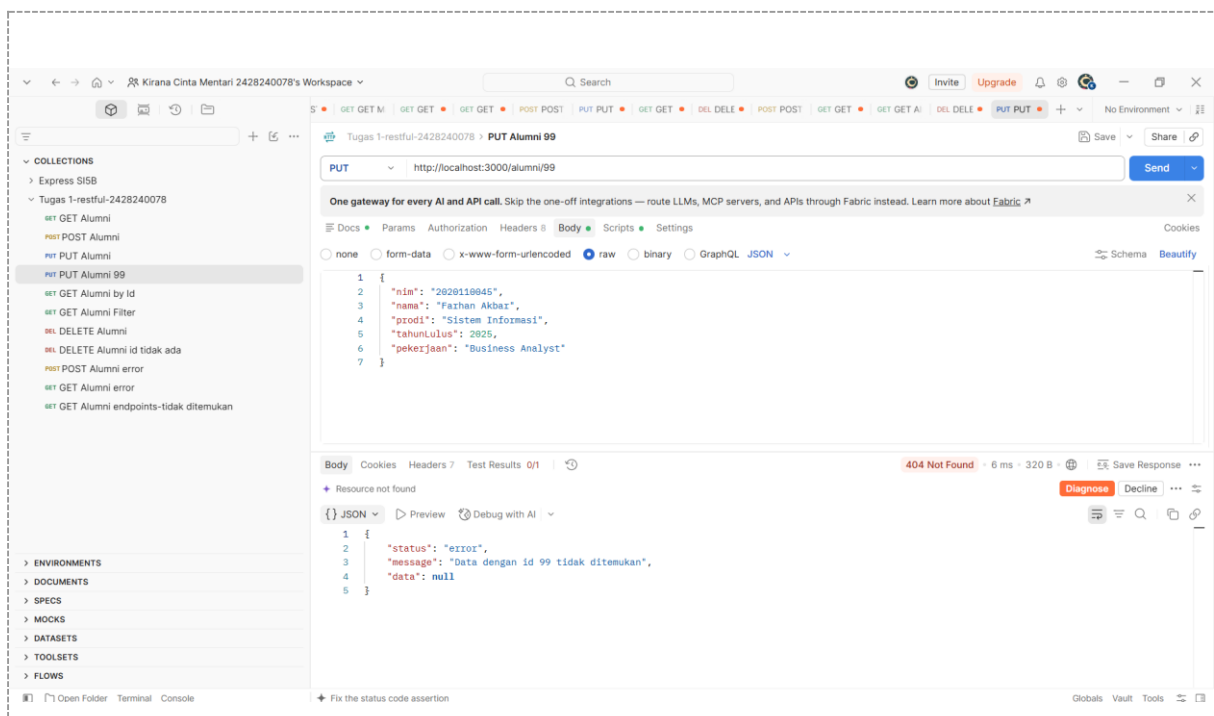
Gambar 5.5. Request 5: POST /alumni dengan body lengkap (201)



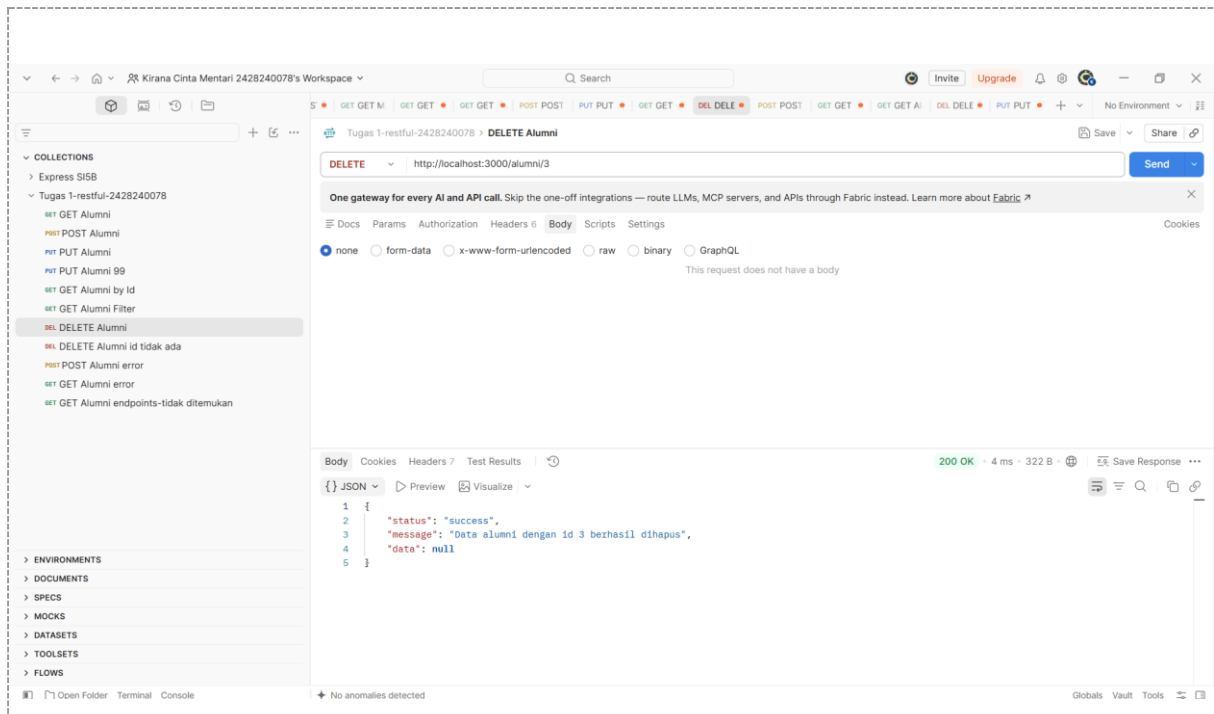
Gambar 5.6. Request 6: POST /alumni dengan field wajib kosong (400)



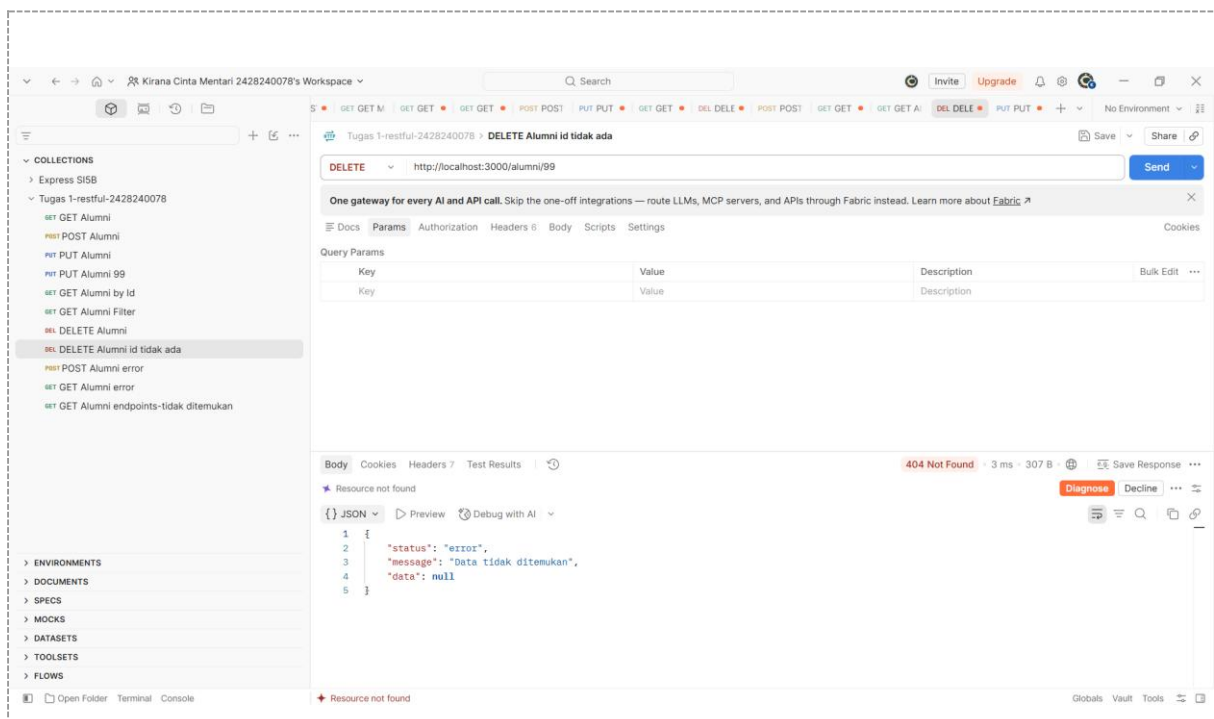
Gambar 5.7. Request 7: PUT /alumni/1 dengan body lengkap (200)



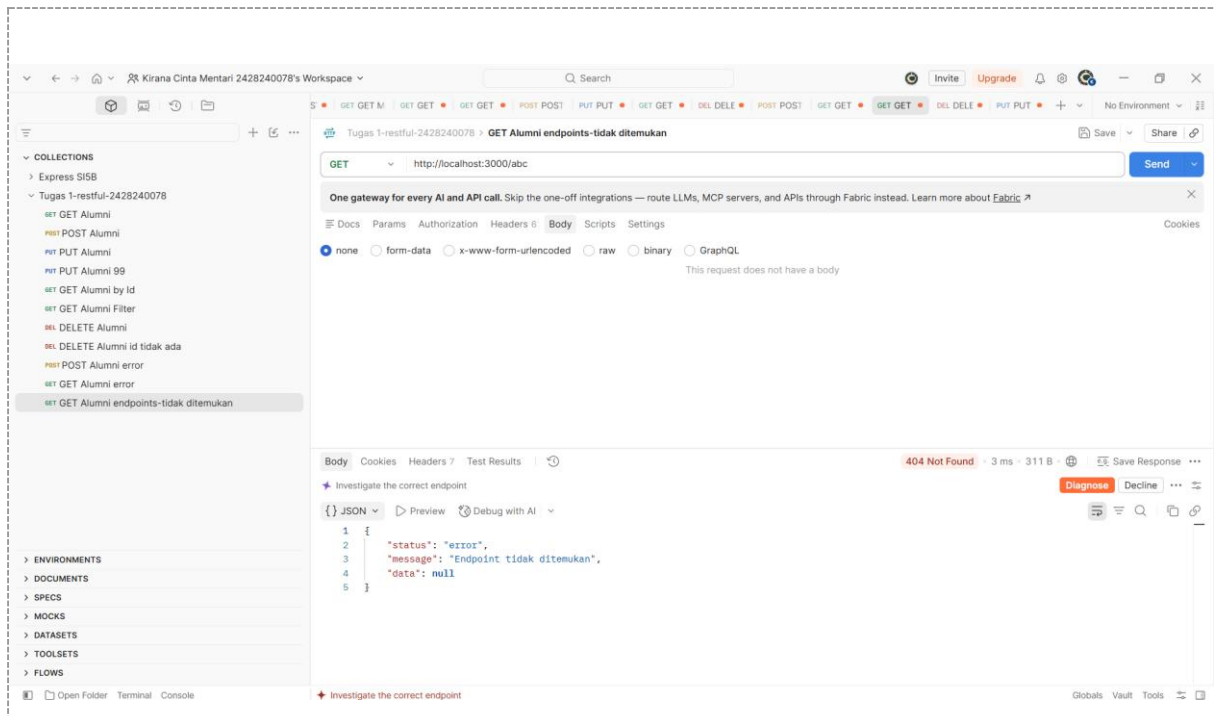
Gambar 5.8. Request 8: PUT /alumni/99 dengan body lengkap (404)



Gambar 5.9. Request 9: DELETE /alumni/3 (200)



Gambar 5.10. Request 10: DELETE /alumni/99 (404)



Gambar 5.11. Request 11: GET /abc, endpoint tidak terdaftar (404)

6. Pembahasan

A. Alasan Pemilihan Method dan Status Code

GET dipakai untuk membaca karena tidak mengubah data, sehingga aman dipanggil berulang. POST dipakai untuk menambah data karena membuat resource baru dan id dibuat oleh server. PUT dipakai untuk mengganti seluruh data sehingga semua field wajib dikirim ulang. DELETE dipakai untuk menghapus data pada id tertentu.

Status 200 dipakai untuk GET, PUT, dan DELETE yang berhasil. Status 201 dipakai pada POST karena ada resource baru yang dibuat. Status 400 dipakai saat validasi gagal karena kesalahan ada pada data kiriman klien. Status 404 dipakai saat id atau endpoint tidak ditemukan.

B. Kendala dan Solusi

No	Kendala	Solusi
1	terdapat karakter yang tidak sengaja terketik pada route POST sehingga server melempar error dan pesan validasi tidak muncul, melainkan pesan error body JSON.	Memeriksa kembali kode dan menghapus karakter tersebut.
2	Middleware 404 salah letak: middleware catch-all sempat berada di dalam route DELETE sehingga URL yang tidak terdaftar belum dibalas JSON.	Memindahkannya ke bawah semua route.
3	node_modules ikut ter-add ke Git: terjadi karena .gitignore belum sesuai.	Mengisi .gitignore (node_modules/ dan .env) dan menghapus node_modules dari staging dengan <code>git rm -r --cached</code> .

7. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. Project RESTful API alumni berhasil dibuat dengan Express.js dalam satu file app.js, dapat dijalankan secara lokal dan di Vercel, serta diunggah ke repository GitHub dengan commit bertahap.
2. Enam endpoint (GET semua, GET by id, POST, PUT, DELETE, dan filter tahunLulus) berhasil dibuat dengan penamaan URL /alumni yang sesuai konvensi dan handler 404.
3. Method GET, POST, PUT, dan DELETE berfungsi sesuai tugasnya dengan validasi field wajib, id otomatis, serta penanganan kasus 404 dan 400.
4. Seluruh response berformat JSON: GET berupa data langsung, sedangkan POST, PUT, DELETE, dan error menggunakan struktur status, message, dan data dengan status code yang tepat.

Daftar Pustaka

Bray, T. (Ed.). (2017). *The JavaScript Object Notation (JSON) Data Interchange Format* (RFC 8259). Internet Engineering Task Force.

Express.js. (2025). Express documentation. <https://expressjs.com>

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine]..

Fielding, R., Nottingham, M., & Reschke, J. (2022). RFC 9110: HTTP Semantics. IETF.

MDN Web Docs. (2025). HTTP response status codes. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>

Lampiran

Link repository GitHub:

[\[kiranacintamentari2428240078/tugas1-restful-2428240078\]](https://github.com/kiranacintamentari2428240078/tugas1-restful-2428240078)

Link deploy Vercel:

[\[https://tugas1-restful-2428240078.vercel.app/\]](https://tugas1-restful-2428240078.vercel.app/)

Riwayat commit (git log --oneline):

```
PS C:\Users\cinta\tugas1-restful-2428240078> git log --oneline main
6913cae (HEAD -> main, origin/main) Remove node_modules from repository
db1f3dc Update app.js
863e764 update package.json
:
6913cae (HEAD -> main, origin/main) Remove node_modules from repository
db1f3dc Update app.js
863e764 update package.json
7a5cb06 export app dan listen hanya saat lokal
221ee67 tambah POST dan validasi
bdd49bb tambah route GET alumni
d74b574 init project express
69becf1 Add Vercel deployment link
4c22a7a Update README
32c9c5c Initial project setup
74645ae Initial project setup
```

Gambar 1. Hasil perintah git log --oneline